Module 10 — Data engineering systems, cloud and Spark foundations

Learning outcomes

Explain batch vs streaming, orchestration, distributed processing and basic cloud architecture without overstating hands-on experience.

1. Batch and streaming

Batch processes bounded data on a schedule. Streaming processes events continuously or near-real-time. Streaming adds event time vs processing time, late events, ordering, checkpointing and replay complexity.

2. Orchestration

A DAG describes task dependencies. Orchestrators schedule, retry, monitor and backfill tasks; they do not necessarily perform the heavy data transformation. Know task dependency, retry, timeout, sensor, schedule and idempotency.

3. Cloud building blocks

• Object storage for raw files and data lake zones.

• Managed databases/warehouses for query and serving.

• Compute services for transformation.

• Identity/access management and secret storage.

• Logging, metrics, alerting and network controls.

Learn concepts first; AWS, Azure and GCP product names vary.

4. Spark fundamentals

Spark DataFrames are distributed tabular abstractions. Transformations are lazy; actions trigger execution. Shuffles move data across partitions and are often expensive. Skew means some partitions contain much more data than others. Broadcast joins can help when one side is small enough. Cache only when reused and measured.

5. File and table formats

CSV is simple but weakly typed and verbose. Parquet is columnar and compressed, suited to analytical scans. Delta Lake, Iceberg and Hudi add table metadata/transaction and evolution capabilities over data files; details differ.

6. Partitioning

Partition on columns used for common pruning and with sensible cardinality. Too many tiny partitions/files increase metadata and scheduling overhead. Partitioning is not the same as indexing or sharding.

7. Security and governance

Least privilege, secret management, encryption, PII minimisation, retention, audit logs, lineage and data ownership. Never put credentials in Git or logs.

Exercises with answer guidance

1. Design a 15-minute latency event pipeline. Discuss broker, consumer, checkpoint, event-time windows and late events.

2. A Spark job is slow after a join. Inspect shuffle, skew, partition counts, join strategy and file sizes.

3. Why can 50,000 tiny Parquet files be worse than fewer larger files? Metadata and task overhead.

4. What happens if an orchestrated task retries after writing output? Make output idempotent or use atomic publish.

5. Explain how to backfill 3 months without duplicating current data.

6. What metrics would alert you before a 7 AM freshness SLA is missed?

Interview boundary

If you studied Spark but did not use it in a project, say “I understand the fundamentals and have practised…” rather than claiming production Spark experience.

External practice for this module

1. DataTalksClub Data Engineering Zoomcamp — work through selected Docker, cloud, orchestration, Spark and warehouse labs.
2. Apache Spark — Quick Start — run DataFrame transformations and inspect lazy evaluation.
3. Apache Airflow — Tutorials — create a small DAG with dependencies, retries and task logging.
4. AWS Skill Builder — learn cloud fundamentals, IAM, S3 and managed data services; use free learning content where available.
5. Databricks Academy — use introductory lakehouse and Spark learning resources.

Applied assignment: build or simulate a daily batch pipeline with raw files, validation, transformation, a warehouse table, run metadata and a rerun test. Then explain how the architecture changes for streaming.

Exit requirement: explain batch vs streaming, event time, checkpointing, shuffle, skew, partition sizing, object storage, IAM and recovery after partial failure. Do not claim production experience from completing tutorials.